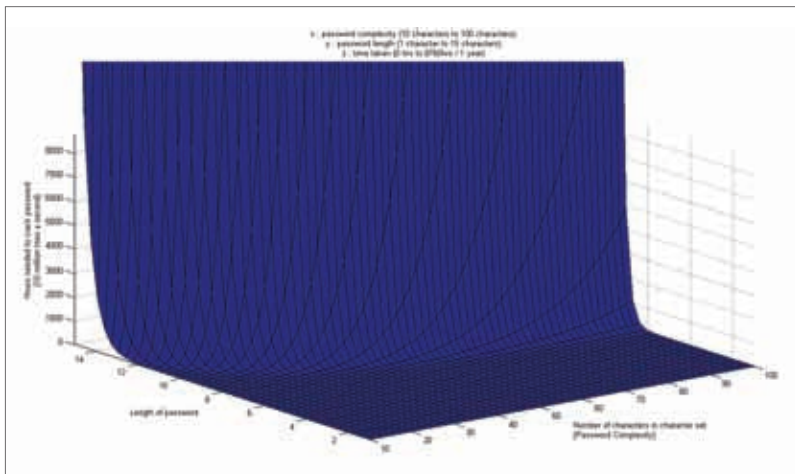




in significantly less time. A real algorithm would take a more probabilistic approach which will check more commonly occurring combinations first. More often than not a password will simply be composed of alphabets and numbers, which decreases our radix from 95 to 62 ($26 + 26 + 10$).

Even so, since the time required guessing a password increases exponentially as the number of symbols in the password increase, we are only buying ourselves enough time for an extra character or two. How about we improve the odds?

In most cases, the passwords people choose for their system will be based on dictionary words. Here of course dictionary doesn't mean an English dictionary, but just a list of words in popular usage which might include words from other languages, common names, slang and 1337speak. This list of words is then used for guessing the password instead of checking each and every combination. As you can imagine, this list will be considerably smaller than all possible combinations, and password length doesn't matter as much as a password's use of dictionary terms. In fact, it might not even matter if the password uses special characters, if such a case is anticipated by the dictionary (for example "cain&abel", "catch-22").

What if, however, the person is using a password which is a combination of dictionary terms with some random characters?

A pure dictionary attack might fail in such a scenario; however, we can use a combination of dictionary and brute force to generate password guesses "near" dictionary words.

The password is not the only point of vulnerability in gaining access to a system / data. When you think about gaining access to something in physical